

How do the runtimes of the algorithms compare in theory?

- These runtimes compare very differently in theory. MergeSort's best case and worst case are $O(n \log n)$. It uses the divide-and-conquer strategy, which splits the array into halves until we reach single elements, then merges them back into order.
- QuickSort uses the divide-and-conquer strategy by using partitions. For QuickSort with Lomuto partitioning, its best case is $O(n \log n)$. This algorithm uses Lomuto partitioning, which works by using the first element as the pivot, and two pointers that both move from left to right. The result is that the smaller elements are pushed to the left of the pivot, and any greater elements are on the right. Though the worst case for QuickSort with Lomuto partitioning is $O(n^2)$ when the partition process is unbalanced, meaning that the array is already sorted or reverse sorted, or all elements are equal.
- QuickSort with Hoare partitioning, its best case is $O(n \log n)$. This algorithm uses Hoare partitioning, which uses the first element as the pivot, and two pointers starting at both ends of the array. These pointers move toward each other, finding mismatched pairs from both ends and swapping them until the pointers cross. The worst case for QuickSort with Hoare partitioning is $O(n^2)$ when the array is already sorted or reverse sorted.

From your experiments, how do their running times in practice compare?

- In practice, Merge sort is consistently the worst algorithm. Though it is close between the Hoare Sort and the Lomuto Sort, in practice, the Lomuto Sort was repeatedly slightly faster than the Hoare Sort.

Why do you think that is the case?

- In theory, Hoare's Sort would typically require fewer swaps due to it using two pointers working their way towards the center. However, Lomuto's algorithm is a bit simpler to run, requiring only one for loop and one pointer, whereas Hoare's requires two while loops and two pointers, which may contribute to more overhead, especially since we are using Python, a high-level language.